

JeevaLife

React + Firebase 2-Day MVP Development Specification

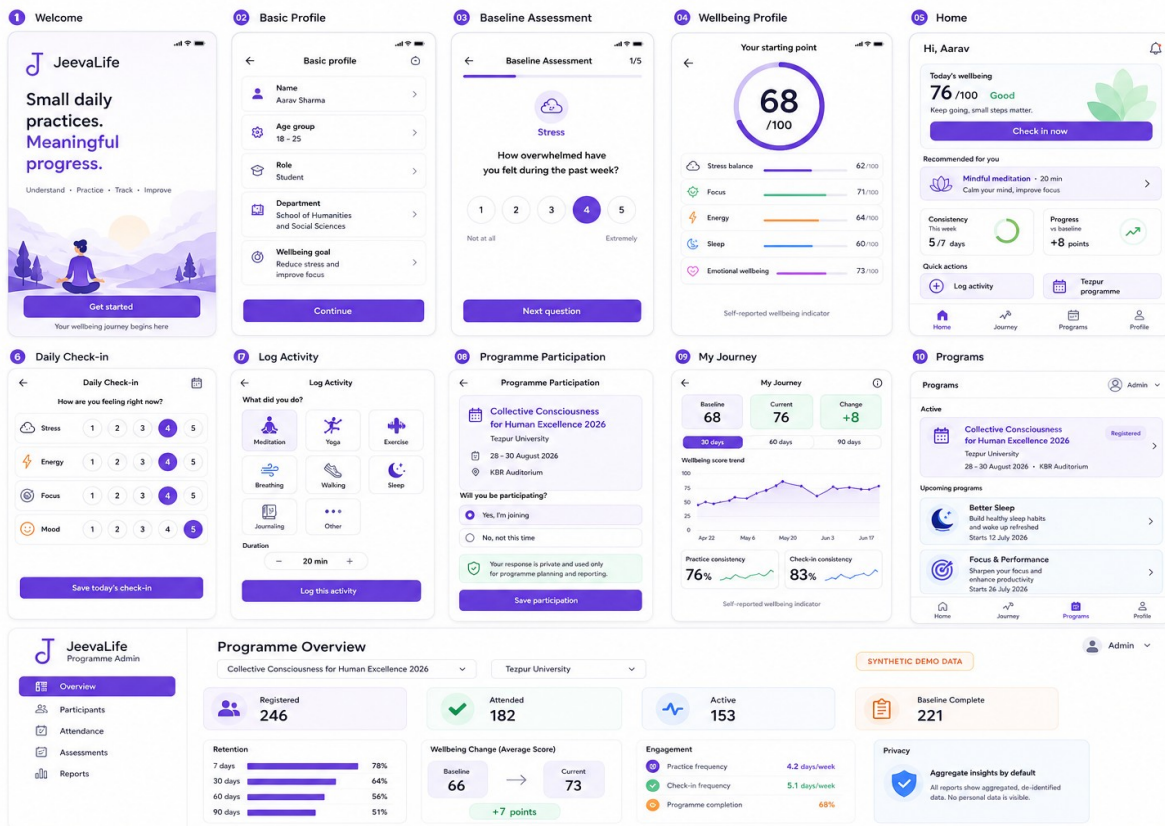
A build-ready guide for frontend and backend developers covering the visual system, screen behavior, architecture, Firestore model, security, wellbeing logic, reporting rules, demo data, testing, and delivery sequence.

2 DAYS

MOBILE-FIRST

LIGHT MODE

TEZPUR DEMO



Design reference supplied by the product owner - visual source of truth for V1.

Executive brief

What the team must deliver, and what a successful two-day MVP means.

Product definition

JeevaLife is a general wellbeing platform for the common public. It is not a TM-only application. Jeevajyoti and Transcendental Meditation are programme types inside a reusable engine: USER -> ASSESS -> ACTIVITY -> TRACK -> CHECK-IN -> PROGRESS -> RETENTION.

Two-day outcome

- A responsive React web app matching the supplied design board at 360-430px mobile widths and remaining usable on tablet and desktop.
- A participant can sign in, complete a profile and baseline assessment, view a computed wellbeing profile, submit a daily check-in, log an activity, record Tezpur programme participation, and view a journey summary.
- An authorised programme admin can open an overview dashboard, see seeded aggregate metrics, inspect participants and attendance, and demonstrate assessment and retention views.
- Firebase Authentication and Firestore persist all real participant actions. Security rules protect personal records and admin-only data.
- The event demo works even before real 30/60/90-day history exists by using clearly labelled synthetic metric snapshots.

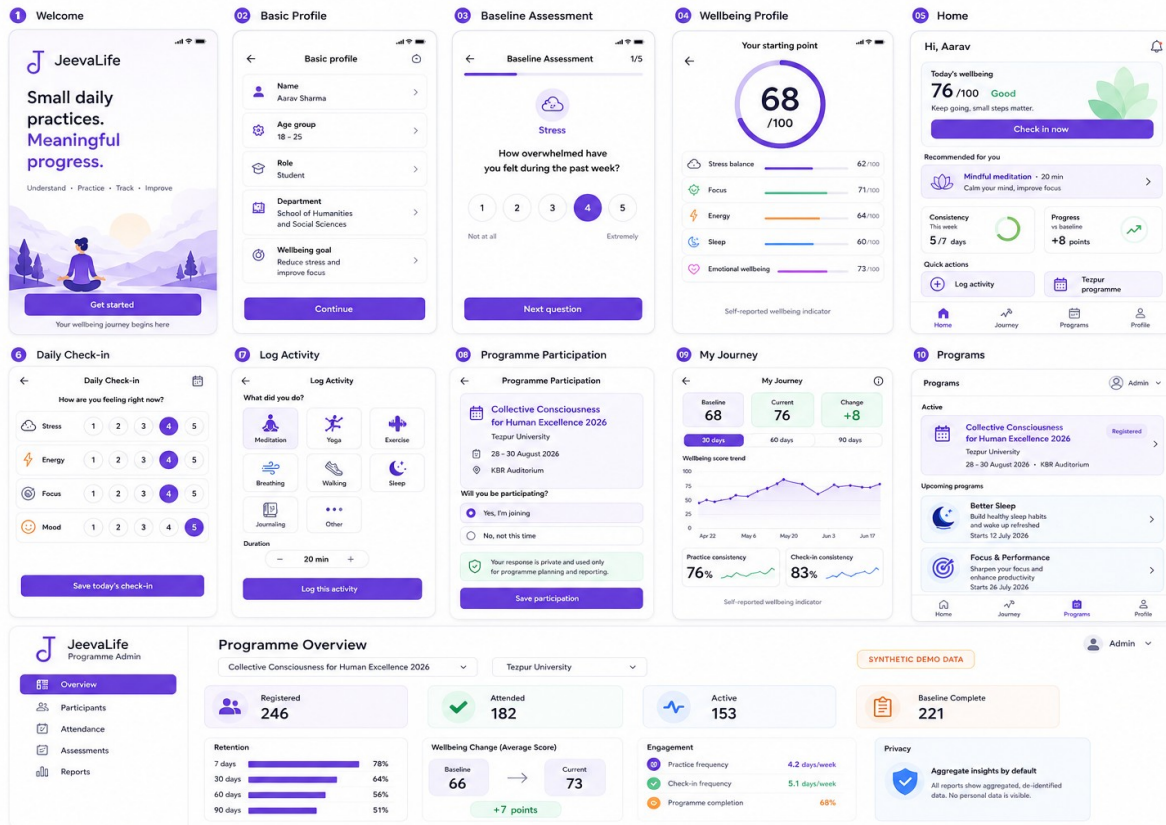
Priority contract

P0	Must work before demo	P1	Add only if P0 is stable
P2	Explicitly defer	RULE	No hidden fake production data

P0 - Build	P1 - If stable	P2 - Do not build in two days
Auth, profile, baseline, result, Home, check-in, activity log	Notification centre UI	AI wellbeing guidance or personalised AI plans
Programme participation, Journey, admin overview	Participant search and CSV export	Community, challenges, marketplace, CMS
Firestore persistence, rules, seed/reset, mobile QA	Bulk attendance and richer empty states	Scheduled retention jobs, research analytics, clinical scoring

Design source of truth

Reproduce the design language and information hierarchy; do not improvise a generic SaaS theme.



Preserve	Interpret carefully	Do not copy blindly
White surfaces, lavender primary action, mint/peach support colours	Illustrations can be replaced by lightweight owned assets	Numbers such as 68, 76, 246 and 182 are demo values
Rounded cards, thin borders, generous spacing, simple line icons	Desktop admin may use denser spacing than participant pages	Dates on future programme cards are placeholders
One primary CTA per screen and mobile bottom navigation	Text may wrap differently at 360px	The design does not define Firestore or scoring rules

Non-negotiable visual decisions

- Light mode only for V1. No dark-mode CSS or system-theme switching.
- Use Inter or a system sans-serif. Functional text must remain legible at 360px.
- Keep shadows extremely soft. Most cards should rely on a 1px border, not floating elevation.
- Use one consistent icon set. Do not mix emoji, outline icons, filled icons, and stock 3D assets.
- Animations are optional, short, and non-blocking. Respect prefers-reduced-motion.
- Do not introduce medical illustrations, diagnosis wording, treatment claims, or a clinical visual identity.

Visual system and responsive rules

Implementation-ready tokens and component behaviour extracted from the design board.

Colour tokens

Token	Value	Use
background	#FFFFFF	Primary participant pages
surface	#FAFAFD	Grouped fields, secondary areas, admin background
primary	#6C4DE6	Primary CTA, selected state, active navigation
primaryDark	#4D35B8	Pressed state and strong purple text
primarySoft	#F0EDFF	Selected cards, progress surfaces, programme hero
mint	#EAF8EF	Positive progress and privacy/success callouts
peach	#FFF1E8	Warm activity and attention accents
text	#202124	Headings and primary labels
muted	#6B7280	Helper text, timestamps, secondary labels
border	#E7E7EE	Cards, fields, dividers

Sizing and shape

Element	Recommended value	Rule
Mobile canvas	360-430px	Design and test here first
Page content padding	16-20px	Never squeeze to screen edge
Card radius	16-24px	Use 18px as default
Button height	48-52px	Full-width primary actions
Touch target	Minimum 44x44px	Includes rating options and back buttons
Body text	14-16px	Never below 12px for functional content
Border	1px #E7E7EE	Default card separation
Motion	120-220ms	Opacity/translate/scale only

Responsive behaviour

- Below 768px: participant experience uses a sticky bottom navigation and single-column cards.
- At 768px and above: centre participant content in a 620px maximum-width shell; do not stretch forms across the screen.
- Admin at 1024px and above: fixed 220-240px sidebar, four metric cards, two-column analytics grid.
- Admin below 900px: collapse sidebar labels into a compact top navigation; tables scroll horizontally without breaking the page.
- No horizontal overflow at 360px. Long programme names must wrap naturally and preserve the CTA.

Product and technical architecture

Keep the platform generic while shipping the Tezpur programme as the first configured use case.

Entity hierarchy

Organisation -> Programme -> Cohort -> Participants -> Activities -> Assessments -> Progress -> Report.
Example: Tezpur University -> Collective Consciousness for Human Excellence 2026 -> University-wide cohort -> participant.

Recommended stack

Layer	Choice	Two-day rule
Frontend	React + TypeScript + Vite	No framework migration during sprint
Routing	React Router	Route guards for onboarding and admin
Styling	Tailwind CSS OR CSS Modules	Pick one; do not mix systems
State	Auth Context + small app context	Do not add Redux for this MVP
Server state	Repository functions; optional TanStack Query	Use only if the team already knows it
Authentication	Firebase Auth - Google and/or email	Google-only is acceptable for demo
Database	Cloud Firestore	Top-level collections for admin queries
Hosting	Firebase Hosting	Use separate dev and production projects if available
Charts	Recharts or lightweight SVG	Only Journey and admin require charts

Frontend route map

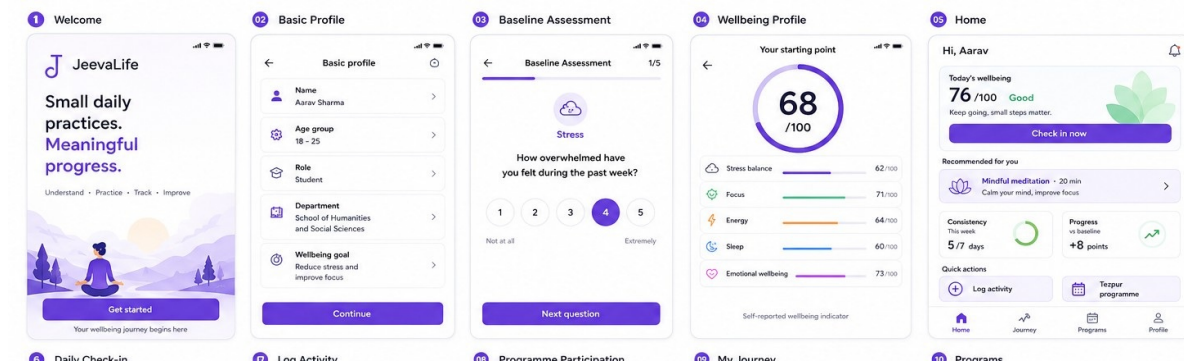
/	Welcome
/auth	Sign in / sign up
/onboarding/profile	Basic profile
/onboarding/assessment	Baseline assessment
/profile/wellbeing	Baseline result
/home	Daily dashboard
/check-in	Daily check-in
/activities/log	Activity logging
/programs	Programme list
/programs/:id	Programme detail / participation
/journey	Progress and trends
/profile	Settings and consent
/admin	Programme overview
/admin/participants	Participants
/admin/attendance	Attendance
/admin/assessments	Assessment summary
/admin/reports	Retention and reports

Guard logic

- Unauthenticated -> /auth.
- Authenticated but profile incomplete -> /onboarding/profile.
- Profile complete but baseline missing -> /onboarding/assessment.
- Participant fully onboarded -> /home.
- Admin routes require an admin custom claim; frontend hiding alone is not security.

Participant screens - onboarding

Screens 01-04 must form one uninterrupted, recoverable journey.



Screen	Purpose	Primary action	Write	Acceptance
01 Welcome	Explain value; enter auth/onboarding	Get started	No user write	Brand mark, headline, calm illustration, one CTA
02 Basic Profile	Collect minimum segmentation data	Continue	users/{uid}	Autosave draft; gender optional; validate name and role
03 Baseline	Collect five self-report dimensions	Next question	assessment draft	One question per view; 1-5 scale; progress 1/5
04 Wellbeing Profile	Show computed starting point	Go to Home	assessments/{id}	Score ring, five bars, non-medical disclaimer

Baseline question contract

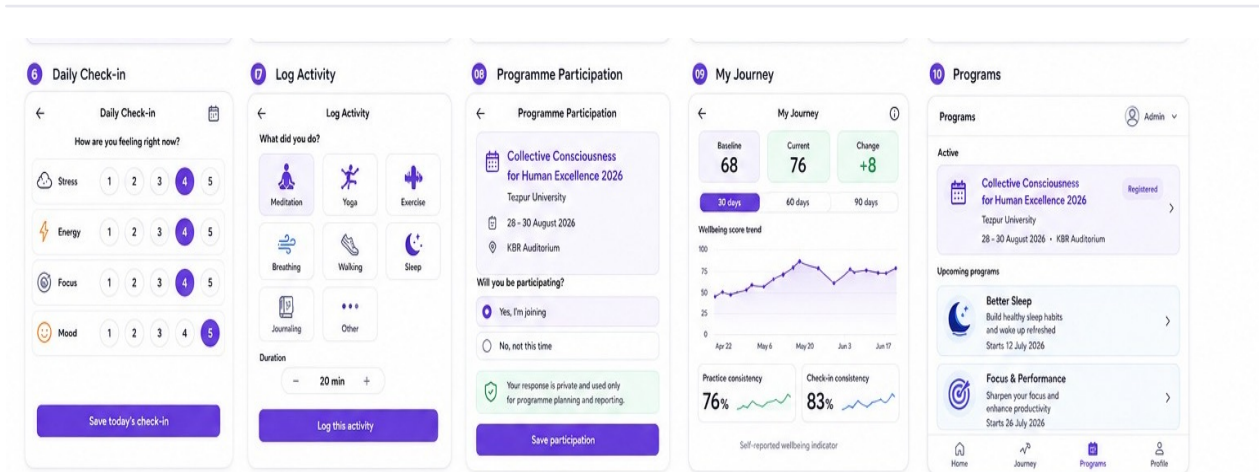
Dimension	Question intent	Direction
Stress	How overwhelmed have you felt during the past week?	Reverse-scored
Focus	How easily could you stay focused on study or work?	Positive
Energy	How steady was your energy through a typical day?	Positive
Sleep	How rested did you feel after waking?	Positive
Emotional wellbeing	How balanced and positive did you feel overall?	Positive

Onboarding behaviour

- Persist each selected answer immediately in local state and optionally in an assessmentDraft document.
- Do not permit result generation until all required answers exist.
- Submitting creates one immutable baseline assessment. Prevent accidental duplicate baseline submissions.
- The values shown in the design are illustrative. Real UI must render the calculated values returned by the scoring utility.
- If the network fails, preserve the draft locally and show a retry state. Never show success before Firestore confirms the write.

Participant screens - daily loop

The daily experience should take less than two minutes: Open -> Check in -> Practice -> Log -> Close.



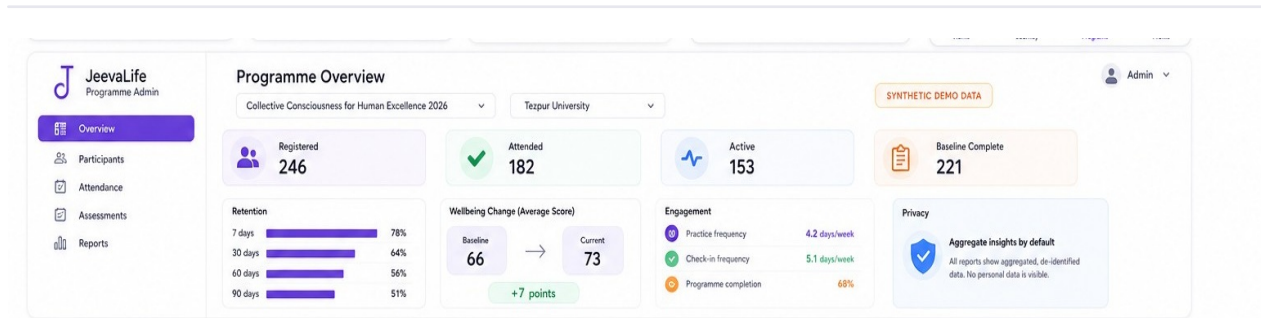
Screen	Must show	Write / read	Critical behaviour
05 Home	Greeting, current indicator, check-in CTA, recommended activity, consistency, quick actions	dashboard summary	One obvious action; loading/error/empty states
06 Daily Check-in	Stress, energy, focus, mood 1-5	checkIns/{uid_dateKey}	One document per local date; explicit submit
07 Log Activity	8 activity types and optional duration	activities/{id}	Duration >= 1; note optional; success after server write
08 Programme Participation	Tezpur name, dates, venue, join choice, privacy note	participations/{uid_program_meld}	Attach to correct programme and cohort
09 My Journey	Baseline/current/change, 30/60/90 selectors, trends, consistency	assessment + metric snapshot	Clearly mark synthetic history in demo
10 Programs	Active event and future programme cards	programmes query	Generic programme engine, not TM-only

Navigation and state

- Bottom navigation is Home, Journey, Programs, Profile. Check-in and Activity Log are contextual actions, not permanent tabs.
- Use a single dateKey helper fixed to the agreed application timezone. Do not mix browser UTC dates with local dates.
- After a successful check-in, Home changes the CTA to a completed/review state. The participant may edit today's check-in if the product owner permits it; do not create duplicates.
- Programme attendance should normally be marked by an admin. The participant screen records join/participation intent; any self-attendance switch must be labelled demo-only.
- The Journey screen must not imply that meditation caused a score change. Use self-reported, change-over-time language only.

Programme admin experience

Responsive desktop-first reporting with aggregate privacy as the default.



View	P0 content	Data source	Privacy rule
Overview	Registered, attended, active, baseline complete	metricSnapshots or demo seed	Aggregate only
Participants	Search, role, cohort, attendance and engagement state	users + participations	Admin-only; no institution public access
Attendance	Session list and participant status	attendance	Admin write; participant can read own
Assessments	Baseline/current averages and completion	assessments aggregate	Never expose raw answers by default
Reports	7/30/60/90 retention and engagement	metricSnapshots	Label synthetic data prominently

Metric definitions - write once, reuse everywhere

Metric	Definition for MVP
Registered	Unique active participations linked to selected programme/cohort
Attended	Participants with at least one attendance record marked present
Active	Participants with a check-in or activity during the last 7 local dateKeys
Baseline complete	Participants with exactly one submitted baseline assessment
Practice frequency	Mean unique activity days per participant per week
Check-in frequency	Mean unique check-in days per participant per week
Programme completion	Participant meets the programme's configured required-session rule
Retention N-day	Eligible participants with a qualifying check-in/activity in the configured N-day observation window

Two-day implementation decision

Do not calculate historical 30/60/90-day retention from nonexistent event data. Seed a metricSnapshots document, show a visible SYNTHETIC DEMO DATA badge, and replace the snapshots with scheduled aggregation after the event.

Frontend implementation

Keep UI, domain logic, and Firebase calls separate so two developers can work without collisions.

Recommended source structure

```
src/
  app/                providers, router, guards
  assets/             owned illustrations and icons
  components/         Button, Card, BottomNav, Scale, Toast
  features/
    auth/             auth pages and hooks
    onboarding/       profile, questions, result
    dashboard/        Home summary
    checkin/          daily check-in
    activities/        activity logger
    programmes/       list, detail, participation
    journey/          scores, chart, consistency
    admin/            overview, users, attendance, reports
  services/           Firebase repositories only
  lib/               firebase.ts, dateKey.ts
  domain/            scoring.ts, metrics.ts, validators.ts
  types/             shared TypeScript interfaces
  styles/            tokens and global styles
```

Component contract

Component	Required props / behaviour
Button	variant, loading, disabled, fullWidth, accessible label
ChoiceScale	value 1-5, onChange, lowLabel, highLabel, keyboard input
ScoreRing	score 0-100, label, colour; no hardcoded demo value
MetricCard	label, value, trend, tone; works with loading skeleton
ActivityTile	type, icon, selected, onSelect
ProgrammeCard	programme metadata, participation state, CTA
BottomNav	activeRoute and four fixed destinations
PageState	loading, error, empty and offline variants

Service boundary

```
profileRepository.getCurrent(uid)
profileRepository.update(uid, payload)
assessmentRepository.getBaseline(uid)
assessmentRepository.submitBaseline(uid, answers)
checkInRepository.getForDate(uid, dateKey)
checkInRepository.upsert(uid, dateKey, payload)
activityRepository.create(uid, payload)
programmeRepository.listActive()
participationRepository.join(uid, programmeId, cohortId)
journeyRepository.getSummary(uid, range)
adminRepository.getProgrammeSnapshot(programmeId)
```

- Page components may call repositories/hooks; they must not contain raw Firestore collection paths.
- Scoring and dateKey utilities must be pure functions with unit tests.
- Every async action needs loading, success, and user-safe error states.
- Never expose raw Firebase error strings in the interface.

Firestore data model

Top-level collections simplify authorised admin queries and programme-wide aggregation.

Collection	Document ID	Important fields
users	uid	name, ageGroup, role, department, wellbeingGoal, consent, onboardingStatus, createdAt
organisations	orgId	name, type, active
programmes	programmId	organisationId, name, type, dates, venue, active, completionRule
cohorts	cohortId	programmId, name, institutionUnit, startDate, endDate
participations	uid_programmId	userId, programmId, cohortId, status, joinedAt
assessments	auto ID	userId, programmId?, type, answers, dimensions, score, submittedAt
checkIns	uid_dateKey	userId, dateKey, stress, energy, focus, mood, createdAt, updatedAt
activities	auto ID	userId, dateKey, type, durationMinutes, note?, occurredAt, createdAt
attendance	uid_sessionId	userId, programmId, cohortId, sessionId, status, markedBy, markedAt
metricSnapshots	programmId_range	programmId, range, metrics, synthetic, generatedAt
notifications	auto ID	userId, type, title, body, readAt, createdAt

TypeScript interface excerpt

```
type WellbeingAnswers = {
  stress: 1 | 2 | 3 | 4 | 5;
  focus: 1 | 2 | 3 | 4 | 5;
  energy: 1 | 2 | 3 | 4 | 5;
  sleep: 1 | 2 | 3 | 4 | 5;
  emotional: 1 | 2 | 3 | 4 | 5;
};

type DailyCheckIn = {
  userId: string;
  dateKey: string; // YYYY-MM-DD in agreed timezone
  stress: 1 | 2 | 3 | 4 | 5;
  energy: 1 | 2 | 3 | 4 | 5;
  focus: 1 | 2 | 3 | 4 | 5;
  mood: 1 | 2 | 3 | 4 | 5;
};
```

- Use serverTimestamp() for authoritative createdAt/updatedAt fields.
- Store dateKey separately for daily uniqueness and indexing.
- Keep raw answers separate from calculated dimensions and overall score.
- Do not allow the participant client to write metricSnapshots, roles, or attendance markedBy fields.
- Never store admin privilege inside a participant-editable users document.

Authentication, consent, and Firestore security

Security rules are part of the MVP definition, not a post-demo enhancement.

Authentication

- Enable Google authentication for the fastest reliable event demo. Add email/password only if already configured.
- On first login, create users/{uid} with onboardingStatus: profile_pending.
- Assign admin access with Firebase custom claims using a trusted Admin SDK workflow. Never let the React app assign roles.
- Sign-out clears user-scoped caches and returns to /auth.

Consent model

Consent	Default	Meaning
Terms/privacy acceptance	Required	Needed before first wellbeing submission
Aggregate programme reporting	Required programme purpose	De-identified counts and averages
Identifiable institutional sharing	Off	Requires explicit separate consent
Research use	Off	Separate future consent; not bundled into MVP

Rules skeleton - adapt and test before deployment

```
function signedIn() { return request.auth != null; }
function owner(uid) { return signedIn() && request.auth.uid == uid; }
function admin() { return signedIn() && request.auth.token.admin == true; }

match /users/{uid} {
  allow create: if owner(uid);
  allow read, update: if owner(uid) || admin();
  allow delete: if false;
}
match /checkIns/{id} {
  allow create: if owner(request.resource.data.userId);
  allow read: if owner(resource.data.userId) || admin();
  allow update: if owner(resource.data.userId)
    && request.resource.data.userId == resource.data.userId;
  allow delete: if false;
}
match /activities/{id} {
  allow create: if owner(request.resource.data.userId);
  allow read: if owner(resource.data.userId) || admin();
  allow update, delete: if false;
}
match /metricSnapshots/{id} { allow read: if admin(); allow write: if false; }
```

Required security tests

Participant A cannot read or write Participant B's profile, check-ins, activities, answers, or attendance. A normal participant cannot access admin routes or metricSnapshots. An admin can read only the data needed for authorised programmes. Run emulator tests if time allows; at minimum run manual deny tests before handoff.

Wellbeing, consistency, and retention logic

Keep formulas deterministic, documented, and independent of presentation components.

Baseline scoring

Simple V1 scoring rule

Positive dimensions: rating x 20. Stress balance: (6 - stress rating) x 20. Overall score: rounded mean of Stress balance, Focus, Energy, Sleep, and Emotional wellbeing. The design's exact numbers are illustrative and must not be hardcoded.

```
const positive = (rating: number) => rating * 20;
const stressBalance = (rating: number) => (6 - rating) * 20;

const dimensions = {
  stress: stressBalance(answers.stress),
  focus: positive(answers.focus),
  energy: positive(answers.energy),
  sleep: positive(answers.sleep),
  emotional: positive(answers.emotional),
};

const score = Math.round(
  Object.values(dimensions).reduce((a, b) => a + b, 0) / 5
);
```

Current score and trends

- Baseline score is the first submitted baseline assessment.
- Current score is the most recent submitted reassessment, not a mixture of attendance, activity count, or streak points.
- Daily check-ins create a separate trend based on mood/focus/energy and reverse-scored stress. Label it Daily wellbeing trend, not assessment score.
- For the two-day demo, seed reassessment and trend records and label the resulting history synthetic.

Consistency

Indicator	Formula
Practice consistency	Unique dateKeys with ≥ 1 activity / elapsed days in selected range x 100
Check-in consistency	Unique submitted check-in dateKeys / elapsed days in selected range x 100
Active participant	At least one check-in or activity during last 7 application dateKeys
Current streak	Consecutive dateKeys ending today or yesterday with a qualifying action

Retention

- Only include participants old enough to be eligible for the selected horizon.
- Define an observation window in one shared metrics utility, for example day N through day N+6 after join.
- Retained means at least one qualifying check-in or activity inside that observation window.
- Snapshot both numerator and eligible denominator. Never calculate percentages from all registered users when some are not yet eligible.
- Final retention definitions require product-owner approval before real institutional reporting.

Seeded demo and Tezpur configuration

The event presentation must be repeatable without pretending historical data is real.

Programme seed

```
{
  id: "collective-consciousness-2026",
  organisationId: "tezpur-university",
  name: "Collective Consciousness for Human Excellence 2026",
  type: "JeevaJyoti Event",
  venue: "KBR Auditorium",
  startDate: "2026-08-28",
  endDate: "2026-08-30",
  active: true
}
```

Synthetic admin snapshot

Metric	Demo value	UI label
Registered	246	Synthetic demo data
Attended	182	Synthetic demo data
Active	153	Synthetic demo data
Baseline complete	221	Synthetic demo data
7/30/60/90 retention	78/64/56/51%	Synthetic demo data
Baseline -> current	66 -> 73	Self-reported; synthetic

Reset strategy

- Use a separate Firebase dev project or a demo namespace. Do not mix synthetic participants with future production participants.
- Create one deterministic seed script using Firebase Admin SDK. Running it twice should update known IDs rather than duplicate records.
- Provide a reset-demo script that deletes only explicit demo document IDs. Never perform broad recursive deletion.
- Use synthetic names and non-identifying email addresses. Do not repurpose real participant data for the demo.
- Rehearse the flow: sign in -> profile -> baseline -> result -> check-in -> activity -> programme -> journey -> admin.

Date clarification

The product handoff mentions a 27 August demonstration, while the programme proposal lists sessions on 28-30 August 2026 at KBR Auditorium. Configure the programme for 28-30 August and treat 27 August as the presentation/demo date unless Rahul confirms otherwise.

Two-day build strategy - Day 1

Foundation, onboarding, Firebase writes, and the complete daily loop.

Time	Frontend developer	Backend developer	Integration checkpoint
09:00-10:00	Create Vite React TS app; tokens; router; responsive shell	Create Firebase project; Auth; Firestore; environments	Both agree types, collection names and route map
10:00-12:00	Welcome, auth UI, Basic Profile	Auth provider, create user document, base rules	Login -> profile persists
12:00-14:00	Baseline question flow and progress	Assessment repository and scoring utility	Submit baseline once; render result
14:00-16:00	Home and Daily Check-in	dateKey utility; check-in upsert; indexes	One check-in per day; Home updates
16:00-18:00	Activity grid, duration, success state	Activity create/read; validation and rules	Logged activity appears in summary
18:00-20:00	Responsive cleanup and loading/error states	Security deny tests; seed programme/config	Full participant loop on mobile
20:00-21:00	Fix integration defects only	Fix integration defects only	Day 1 gate and commit

Day 1 gate - do not continue until these pass

- User can sign in and sign out.
- Profile survives refresh and cannot overwrite another user's profile.
- Baseline validates all five answers, submits once, and renders calculated values.
- Check-in writes one dateKey record and handles repeat submissions correctly.
- Activity log persists type and duration and shows success only after confirmation.
- Core screens have no horizontal overflow at 360px.

Parallel work rule

Avoid Git collisions

Frontend owns app/components/features/styles. Backend owns services/lib/domain rules, Firebase configuration, seed scripts, and security tests. Shared TypeScript types are agreed in the first hour; later changes require both developers to update their integration branch before continuing.

Two-day build strategy - Day 2

Programme participation, Journey, admin, privacy, polish, deployment, and rehearsal.

Time	Frontend developer	Backend developer	Integration checkpoint
09:00-11:00	Programs, programme detail, Tezpur participation	Programmes/cohorts/participations repositories	Join state persists and reloads
11:00-13:00	My Journey cards, selectors and trend chart	Journey summary; metric snapshot reads	Real baseline + labelled synthetic history
13:00-15:00	Admin shell, overview metrics and retention bars	Custom admin claim; admin repository	Normal user denied; admin loads snapshot
15:00-16:30	Participants/attendance basic views	Attendance writes; admin rules; indexes	Attendance updates dashboard state
16:30-18:00	Accessibility, empty/error/offline states	Rules audit, seed/reset, data validation	Primary flows pass on phone and desktop
18:00-19:00	Production build and visual polish	Deploy Firebase config/rules/hosting	Clean production deployment
19:00-20:00	Rehearse participant demo	Rehearse admin demo and reset	Record fallback screenshots/video
20:00-21:00	Bug-fix buffer only	Bug-fix buffer only	Final handoff

Scope-cut order if delayed

- First cut notifications and exports.
- Then cut editable participant tables and bulk attendance; keep the admin overview.
- Then keep Journey history fully synthetic but visibly labelled.
- Never cut authentication, security rules, profile/baseline persistence, check-in, or activity logging.
- Never present local-only mock data as a working production backend.

Recommended ownership

Owner	Accountability
Frontend	Visual fidelity, routes, components, accessibility, responsive behaviour, client validation, repository integration
Backend	Firebase projects, Auth, Firestore model, repositories, rules, indexes, admin claim, seed/reset and security testing
Shared	Types, scoring definitions, dateKey timezone, metric definitions, QA checklist and demo script
Product owner	Final questions, consent wording, retention definition, event dates, completion rule and approval of synthetic data label

QA and acceptance criteria

A screen is not complete until it works, persists, handles errors, and matches mobile behaviour.

Area	Acceptance criteria
Auth	Sign in, refresh, and sign out work. Admin routes reject normal users.
Profile	Required fields validate; gender remains optional; saved profile reloads.
Assessment	Incomplete submit blocked; duplicate baseline prevented; result matches scoring utility.
Check-in	One record per dateKey; update policy defined; no false success during network failure.
Activity	Type and duration persist; invalid duration rejected; duplicate tap does not double-write.
Programme	Tezpur metadata is correct; join state attaches to programme/cohort.
Journey	Baseline/current/change are internally consistent; synthetic history visibly labelled.
Admin	Only custom-claim admin can access; snapshot badge remains visible.
Privacy	Institution-facing view is aggregate; identifiable sharing is off by default.
Responsive	No overflow at 360px; participant shell centres on desktop; admin table remains usable.
Accessibility	Keyboard focus visible; controls labelled; colour is not the only state indicator.
Reliability	Loading, error, empty and offline states exist for every async primary screen.

Minimum test matrix

Device / state	Test
360px Android-sized viewport	Complete participant journey with no horizontal scroll
390-430px viewport	Compare hierarchy, spacing, cards, navigation and CTA to design board
Desktop 1366px	Admin sidebar, metrics and tables fit without clipping
Slow network	Loading skeleton, disabled submit and recoverable error
Offline after input	Draft remains; unsaved state is communicated
Normal user on /admin	Denied and redirected
User A queries User B	Firestore denies request
Duplicate submission	Baseline/check-in/activity protections behave as specified

Build and deployment checks

- npm run build completes with no TypeScript errors.
- No Firebase credentials beyond public web config are committed; service-account keys never enter the React project.
- Firestore indexes required by production queries are deployed.
- Production Firebase project does not contain accidental synthetic user identities unless isolated and labelled.
- A demo reset path and fallback screenshots/video are available before travel to Tezpur University.

Final handoff checklist

What the development team must return after the two-day sprint.

Deliverable	Required evidence
React source	Documented project structure, environment setup, build commands
Firebase configuration	Auth providers, Firestore rules, indexes and project identifiers documented
Data contract	Types and collection fields match this document
Seed/reset	Deterministic programme/snapshot seed and safe demo reset
Security	Participant isolation and admin deny tests recorded
QA	Mobile and desktop acceptance checklist signed off
Deployment	Working production URL and known limitations
Demo script	Participant and admin sequence rehearsed end-to-end

Decisions the product owner must confirm before production use

- Is 27 August the presentation date while 28-30 August are the programme dates?
- Which authentication methods will real participants use?
- Is a participant allowed to edit today's check-in after submission?
- Is attendance admin-only, or can participants self-report attendance?
- What counts as programme completion - all sessions, a minimum number, or a manual status?
- What exact observation window defines 7/30/60/90-day retention?
- What consent language has legal/product approval for aggregate and identifiable reporting?

Explicit V1 exclusions

- AI guidance, personalised AI plans, clinical assessment, treatment advice or causal claims.
- Community feeds, gamified challenges, leaderboards, coins, marketplace or expert booking.
- Complex CMS, advanced research analytics, automated publication reports or university-wide ERP integration.
- A TM-only brand architecture. New programme types must fit the same generic programme model.

Final engineering principle

Build the engine, not the entire ecosystem. When a choice is unclear, choose the option that is simpler, faster, more accessible, easier to maintain, privacy-preserving, and demonstrably reliable within two days.

End of specification